

Wording for Networking PDTS ballot comments 026 (GB-15) and 027 (GB-16)

Introduction

This paper presents proposed changes in response to two of the National Body comments for PDTS 19216 from [N4643](#).

The comments were discussed by LEWG in Toronto and there were concerns about dropping support for Windows 8.0 and earlier. This paper proposes the changes for discussion.

Changes

NB comment 026 (GB-15)

Consider adding `release()` member functions to `basic_socket` and `basic_socket_acceptor`. Historically, Asio has not provided this facility as it could not be portably implemented using the preferred OS-specific mechanisms. Specifically, on Windows, once a socket was associated to an I/O completion port it could not be disassociated. This means that the socket could not be truly released for arbitrary use by the user. However, as of Windows 8.1, the ability to disassociate a socket is available (via the kernel API `NtSetInformationFile`). This means that `release()` can be portably implemented.

Proposed change: Add the following member functions to both `basic_socket` and `basic_socket_acceptor`:

```
native_handle_type release();  
native_handle_type release(error_code& ec);
```

with effects that any pending asynchronous operations are cancelled and ownership of the native handle is transferred to the caller.

Chris Kohlhoff added:

This has been implemented on the master branch of standalone Asio. It provides a counterpart to `assign()`, and allows native sockets to be released for use by non-Networking TS code.

NB comment 027 (GB-16)

Consider adding constructors to `basic_socket` and `basic_socket_acceptor` to move a socket to another `io_context`. Historically, Asio has not provided this facility as it could not be portably implemented using the preferred OS-specific mechanisms. Specifically, on Windows, once a socket was associated to an I/O completion port it could not be disassociated. This means that the socket could not be moved from one `io_context` to another. However, as of Windows 8.1, the ability to disassociate a socket is available (via the kernel API `NtSetInformationFile`). This means that the ability to move sockets between `io_contexts` can be portably implemented.

Proposed change: Add constructors to `basic_socket` (and to derived classes `basic_stream_socket`, `basic_stream_socket`), and to `basic_socket_acceptor`, e.g.: `basic_socket(io_context& ctx, basic_socket&& rhs)`; with effect that pending asynchronous operations are cancelled on `rhs`, and then ownership of the underlying socket is transferred to the newly created socket object with the associated `io_context`.

Chris Kohlhoff added:

This would typically be implemented in terms of `release()` above.

It enables portable share-nothing, sharded designs. E.g. an `io_context` per CPU design, where newly accepted sockets are transferred to a different `io_context`/CPU based on some load balancing algorithm.

I explicitly mention "portable" above because `release()` returns `native_handle_type`, which is an implementation defined type.

Discussion

The proposed features seem suitable for an experimental TS, but probably not for the IS while Windows 8.0 and earlier are still in widespread use.

The Networking TS already allows a lot of implementation freedom for non-portable features. The presence of the new `release` member functions is implementation-defined (see the changes to 18.2.3 [socket.reqmts.native] below) and 4.1.1 [conformance.9945] allows unimplementable behaviour to be unsupported:

Implementations are not required to provide behavior that is not supported by a particular operating system.

If moving a socket from one completion port to another is not possible for older Windows systems, the corresponding constructors can throw an error.

Design questions

- Is being unimplementable on a widely-used OS acceptable for the TS?
- Is *Requires: `is_open() == true`* the right choice for `release()` ?
- Do we even want a constructor with such significant side effects (canceling all outstanding async operations, no strong exception safety guarantee)?

Assuming `native_handle()` is provided by the implementation, similar effects can be achieved using just the new `release()` function. e.g. for POSIX-based systems where `native_handle_type` is the file descriptor for the socket:

```
tcp::socket transfer(io_context& ctx, tcp::socket& sock)
{
    tcp::socket newsock(ctx);
    int native = sock.release();
    try {
        newsock.assign(ip::tcp::v4(), native);
    } catch (...) {
        ::close(native);
        throw;
    }
}
```

```
    return newsock;
}
```

However, the constructor can achieve similar effects even without exposing native handles.

Proposed Wording

Changes are relative to [N4656](#).

Modify 18.2.3 [sockets.reqmts.native] paragraph 1 as shown:

Several classes described in this Technical Specification have a member type `native_handle_type`, a member function `native_handle`, and member functions that return or accept arguments of type `native_handle_type`. The presence of these members and their semantics is implementation-defined.

Modify 18.6 [socket.synop] as shown:

```
void assign(const protocol_type& protocol,
            const native_handle_type& native_socket); // see 18.2.3
void assign(const protocol_type& protocol,
            const native_handle_type& native_socket,
            error_code& ec); // see 18.2.3
```

```
native_handle_type release(); // see 18.2.3
native_handle_type release(error_code& ec); // see 18.2.3
```

```
bool is_open() const noexcept;
```

```
[...]
```

protected:

```
// construct / copy / destroy:
```

```
explicit basic_socket(io_context& ctx);
basic_socket(io_context& ctx, const protocol_type& protocol);
basic_socket(io_context& ctx, const endpoint_type& endpoint);
basic_socket(io_context& ctx, const protocol_type& protocol,
            const native_handle_type& native_socket); // see 18.2.3
basic_socket(const basic_socket&) = delete;
basic_socket(basic_socket&& rhs);
template<class OtherProtocol>
    basic_socket(basic_socket<OtherProtocol>&& rhs);
basic_socket(io_context& ctx, basic_socket&& rhs);

~basic_socket();
```

Add to the end of 18.6.1 [socket.basic.cons]:

```
basic_socket(io_context& ctx, basic_socket&& rhs);
```

-15- *Effects:* Constructs a `basic_socket` as if by `basic_socket(ctx)` and then if `rhs.is_open()` is true, `assign(rhs.protocol_, rhs.release())`. If the call to `assign` fails, closes the native socket as if by `POSIX close`.

-16- *Postconditions:* `rhs.is_open() == false`.

-17- *[Note: This constructor depends on functionality which is not implementable on all operating systems, and so may not be usable in portable programs. --end note]*

Add to 18.6.4 [socket.basic.ops]:

-9- *Error conditions:*

- `socket_errc::already_open` -- if `is_open() == true`.

`native_handle_type release();`
`native_handle_type release(error_code& ec);`

-?- *Requires:* `is_open() == true`.

-?- *Effects:* Cancels all outstanding asynchronous operations associated with this socket. Completion handlers for canceled asynchronous operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields `true`.

-?- *Returns:* The native representation of this socket.

-?- *Postconditions:* `is_open() == false`.

-?- *Remarks:* Since the native socket is not closed prior to returning it, the caller is responsible for closing it.

```
bool is_open() const noexcept;
```

-10- *Returns:* A `bool` indicating whether this socket was opened by a previous call to `open` or `assign`.

Modify 18.7 [socket.dgram] as shown:

// construct / copy / destroy:

```
explicit basic_datagram_socket(io_context& ctx);
basic_datagram_socket(io_context& ctx, const protocol_type& protocol);
basic_datagram_socket(io_context& ctx, const endpoint_type& endpoint);
basic_datagram_socket(io_context& ctx, const protocol_type& protocol,
                      const native_handle_type& native_socket);
basic_datagram_socket(const basic_datagram_socket&) = delete;
basic_datagram_socket(basic_datagram_socket&& rhs);
template<class OtherProtocol>
    basic_datagram_socket(basic_datagram_socket<OtherProtocol>&& rhs);
basic_datagram_socket(io_context& ctx, basic_datagram_socket&& rhs);
```

```
~basic_datagram_socket();
```

Modify 18.7.1 [socket.dgram.cons] as shown:

`basic_datagram_socket(io_context& ctx, basic_datagram_socket&& rhs);`

-9- *Effects:* Initializes the base class with `basic_socket(ctx, std::move(rhs))`.

-10- *[Note: This constructor depends on functionality which is not implementable on all operating systems, and so may not be usable in portable programs. --end note]*

Modify 18.8 [socket.stream] as shown:

// construct / copy / destroy:

```
explicit basic_stream_socket(io_context& ctx);
basic_stream_socket(io_context& ctx, const protocol_type& protocol);
```

```

basic_stream_socket(io_context& ctx, const endpoint_type& endpoint);
basic_stream_socket(io_context& ctx, const protocol_type& protocol,
                    const native_handle_type& native_socket);
basic_stream_socket(const basic_stream_socket&) = delete;
basic_stream_socket(basic_stream_socket&& rhs);
template<class OtherProtocol>
    basic_stream_socket(basic_stream_socket<OtherProtocol>&& rhs);
basic_stream_socket(io_context& ctx, basic_stream_socket&& rhs);

~basic_stream_socket();

```

Modify 18.8.1 [socket.stream.cons] as shown:

```

basic_stream_socket(io_context& ctx, basic_stream_socket&& rhs);

```

-9- Effects: Initializes the base class with `basic_socket(ctx, std::move(rhs))`.

-10- [Note: This constructor depends on functionality which is not implementable on all operating systems, and so may not be usable in portable programs. --end note]

Modify 18.9 [socket.acceptor] as shown:

```

void assign(const protocol_type& protocol,
            const native_handle_type& native_acceptor); // see 18.2.3
void assign(const protocol_type& protocol,
            const native_handle_type& native_acceptor,
            error_code& ec); // see 18.2.3

```

```

native_handle_type release(); // see 18.2.3
native_handle_type release(error_code& ec); // see 18.2.3

```

```

bool is_open() const;

```

```

[... ]

```

// construct / copy / destroy:

```

explicit basic_socket_acceptor(io_context& ctx);
basic_socket_acceptor(io_context& ctx, const protocol_type& protocol);
basic_socket_acceptor(io_context& ctx, const endpoint_type& endpoint,
                      bool reuse_addr = true);
basic_socket_acceptor(io_context& ctx, const protocol_type& protocol,
                      const native_handle_type& native_socket);
basic_socket_acceptor(const basic_socket_acceptor&) = delete;
basic_socket_acceptor(basic_socket_acceptor&& rhs);
template<class OtherProtocol>
    basic_socket_acceptor(basic_socket_acceptor<OtherProtocol>&& rhs);
basic_socket_acceptor(io_context& ctx, basic_socket_acceptor&& rhs);

~basic_socket_acceptor();

```

Add to the end of 18.9.1 [socket.acceptor.cons]:

```

basic_socket_acceptor(io_context& ctx, basic_socket_acceptor&& rhs);

```

-15- Effects: Constructs a basic socket acceptor as if by `basic_socket_acceptor(ctx)` and then if `rhs.is_open()` is true, `assign(rhs.protocol, rhs.release())`. If the call to `assign` fails, closes the native acceptor as if by POSIX `close`.

-16- Postconditions: `rhs.is_open() == false`.

-17- [Note: This constructor depends on functionality which is not implementable on all operating systems, and so may not be usable in portable programs. --end note].

Add to 18.9.4 [socket.acceptor.ops]:

-9- *Error conditions:*

- `socket_errc::already_open` -- if `is_open()` == `true`.

`native_handle_type release();`
`native_handle_type release(error_code& ec);`

-?- *Requires:* `is_open()` == `true`.

-?- *Effects:* Cancels all outstanding asynchronous operations associated with this acceptor. Completion handlers for canceled asynchronous operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields `true`.

-?- *Returns:* The native representation of this acceptor.

-?- *Postconditions:* `is_open()` == `false`.

-?- *Remarks:* Since the native acceptor is not closed prior to returning it, the caller is responsible for closing it.

`bool is_open() const noexcept;`

-10- *Returns:* A `bool` indicating whether this acceptor was opened by a previous call to `open` or `assign`.